GROUP ASSIGNMENT 02

Part 1 — Architecture Checkpoint

Scenario C: Omnichannel Commerce Core

VerdeMart Retail — Architectural Evolution of nopCommerce

Course	Software Architectures — Master in Informatics Engineering
Assignment	Group Assignment 02 — Architecture Checkpoint (Part 1)
Scenario	C — Omnichannel Commerce Core
Framework	ADD (Attribute-Driven Design)
Group Size	4 students
Date	May 2026

1. Scenario Choice

1.1 Selected Scenario

We selected Scenario C — Omnichannel Commerce Core.

VerdeMart Retail started with nopCommerce as a standard web storefront. The business has grown to require a unified commerce experience across multiple operational channels: online sales, in-store POS, warehouse execution, shipping coordination, customer support, and back-office finance. nopCommerce must evolve from an isolated web shop into a reliable commerce core that orchestrates and maintains consistency across all these channels.

1.2 Why This Scenario Matters

The business pressure in Scenario C is architecturally honest and realistic. It does not ask for a rewrite — it asks for a system that was designed as a single storefront to absorb cross-channel responsibility gracefully. The key tensions are:

- nopCommerce was not designed as a system-of-systems hub. Its internal coupling makes it fragile when external operational systems disagree, lag, or go offline.
- Stock and order state are shared across channels (web, POS, warehouse). Without coordination, customers can purchase items that are already reserved or shipped elsewhere.
- Surrounding systems (ERP, WMS, shipping carriers) are operationally independent and can become stale, delayed, or temporarily unavailable — but the commerce core must remain useful and coherent during these disruptions.
- Recovery after degradation is as important as normal-path operation. The architecture must show how consistency is restored when systems come back online.

This scenario forces explicit decisions about asynchronous coordination, resilience boundaries, data ownership, and consistency models — exactly the kind of architectural judgment the assignment is designed to test.

2. Current-State Analysis

2.1 nopCommerce Baseline Architecture

nopCommerce is a modular monolith written in C# on ASP.NET Core. Its internal structure follows a layered / onion-style pattern:

Layer	Responsibility
Presentation Layer	Razor-based web UI, controllers, view models. Handles HTTP requests and renders storefront pages.
Application / Service Layer	Business logic: order processing, cart management, pricing, discounts, payment orchestration.
Domain Layer	Core entities: Product, Order, Customer, ShoppingCart, Shipment, Inventory.
Infrastructure Layer	Entity Framework Core, SQL Server, plugin system, caching (in-memory / Redis), email, file storage.
Plugin System	Extension mechanism for payment gateways, shipping providers, tax calculators. Loaded at startup.

2.2 Architectural Seams Relevant to Scenario C

The following parts of nopCommerce are directly implicated in the omnichannel evolution:

- Inventory / Stock module: Stock quantities are stored in a single SQL table (StockQuantityHistory). There is no concept of reserved stock across channels or eventual-consistency inventory. POS and WMS cannot safely share this state without coordination.
- Order processing pipeline: Orders are created synchronously in a single database transaction. There is no outbox, no event publication, and no mechanism to propagate order state to external systems reliably.
- Shipment tracking: nopCommerce models shipments as internal records. It has no webhook receiver or async handler for external carrier status updates.
- No message bus integration: nopCommerce has no built-in support for publishing or consuming domain events. All integration today is synchronous HTTP or direct database access.
- Plugin shipping providers: Shipping plugins (e.g., UPS, FedEx) are called synchronously at checkout. If a carrier API is unavailable, checkout fails entirely.
- Customer and identity: nopCommerce has its own customer store. There is no federation or external identity provider integration out of the box.

2.3 Pressure Points Identified

Critical Pressure Points for Scenario C

P1 — Inventory Race Condition: Web and POS channels can simultaneously reduce stock for the same SKU with no cross-channel reservation mechanism.

P2 — Synchronous ERP Coupling: Any ERP call at order time creates a hard dependency. ERP downtime = checkout failure.

P3 — Stale Stock Visibility: WMS updates (receipts, picks, damage writes) are not propagated back to nopCommerce in near-real-time.

P4 — Carrier API Fragility: Synchronous carrier calls at checkout make shipping rate retrieval a single point of failure.

P5 — No Event Trail: Without domain events, cross-channel state reconciliation after failures is manual and error-prone.

3. Domain and Boundary Model

3.1 Relevant Subdomains

Subdomain	Classification and Ownership
Order Management	Core — Creating, confirming, and tracking orders across channels. Lives inside nopCommerce.
Inventory / Stock	Core — Stock reservation, availability, and reconciliation across web, POS, and WMS.
Fulfillment	Supporting — Warehouse execution, pick/pack/ship. Owned by WMS (OpenBoxes).
ERP / Finance	Supporting — Back-office accounting, purchasing, supplier invoices. Owned by ERPNext.
Carrier / Shipping	Generic — Rate retrieval and tracking. Third-party. Simulated via WireMock.
POS / In-Store	Supporting — In-store sales and stock consumption. OSPOS or equivalent.
Customer & Identity	Supporting — Customer data, authentication. Keycloak + nopCommerce customer model.
Notification	Generic — Email/SMS confirmations. Internal to nopCommerce.

3.2 Bounded Contexts

We identify six bounded contexts for the evolved architecture:

- Commerce Core (nopCommerce monolith): Order lifecycle, cart, pricing, product catalog, customer-facing operations. This is the authoritative source for order state.
- Inventory Context (extracted service): Cross-channel stock reservation and availability. Owns the single source of truth for reservable stock. Communicates via events and a reservation API.
- Fulfillment Context (OpenBoxes WMS): Warehouse execution — picks, packs, shipments. Receives order assignments via message bus. Publishes fulfillment events back to Commerce Core.
- Back-Office Context (ERPNext): Financial records, purchasing, supplier management. Receives order summaries asynchronously. Does not participate in the synchronous checkout path.
- Channel Context (POS / OSPOS): In-store sales. Consumes stock via Inventory Context reservation API. Publishes sales events.
- Carrier Integration Context (WireMock surrogate): Shipping rate retrieval and tracking. Accessed asynchronously via a thin adapter with circuit-breaker protection.

3.3 Context Map

Context Relationships

Commerce Core → Inventory Context: Customer/Supplier (Commerce Core initiates reservations; Inventory Context owns stock truth)
Commerce Core → Fulfillment Context: Published-Language event (OrderConfirmed event triggers WMS pick assignment)
Commerce Core → Back-Office Context: Published-Language event (OrderConfirmed event triggers ERP financial record asynchronously)
Commerce Core → Carrier Context: Anti-Corruption Layer (adapter shields nopCommerce from carrier API volatility)
Channel (POS) → Inventory Context: Customer/Supplier (POS calls Inventory reservation API for in-store sales)
Fulfillment → Commerce Core: Published-Language event (ShipmentDispatched, TrackingUpdated events)

3.4 Ownership of Major Responsibilities

Responsibility	Owner
Order state (authoritative)	Commerce Core (nopCommerce)
Stock availability & reservation	Inventory Context (extracted service)
Warehouse execution	Fulfillment Context (OpenBoxes)
Financial records	Back-Office Context (ERPNext)
In-store sales	Channel Context (OSPOS)
Carrier rates & tracking	Carrier Context (WireMock surrogate)
Domain event bus	RabbitMQ (shared infrastructure, not shared DB)

4. Quality Attribute Scenarios

The following five scenarios drive all major architectural decisions. Each follows the standard QAS format: Source → Stimulus → Environment → Response → Response Measure.

QAS-1 — Availability: ERP Unavailability During Checkout

Source: ERPNext back-office system

Stimulus: ERPNext becomes unavailable (network partition or restart) during peak checkout period

Environment: Normal operating load, 100+ concurrent checkout attempts

Response: Commerce Core completes checkout and publishes OrderConfirmed event to outbox. ERP integration retries asynchronously when ERPNext recovers.

Measure: Checkout success rate remains ≥ 99%. ERP records are eventually consistent within 5 minutes of ERPNext recovery. Zero lost orders.

QAS-2 — Consistency: Cross-Channel Stock Reservation

Source: Simultaneous web customer and in-store POS operator

Stimulus: Both attempt to purchase the last unit of a SKU within the same second

Environment: Normal load, stock = 1 unit, web and POS channels active

Response: Exactly one reservation succeeds. The other receives an immediate out-of-stock response. No overselling occurs.

Measure: Zero oversell incidents under concurrent load test (50 simultaneous reservation attempts for qty=1 item). Reservation API P99 latency ≤ 200ms.

QAS-3 — Resilience: Carrier API Degradation at Checkout

Source: Shipping carrier API (WireMock surrogate configured for 5s delay / 503 errors)

Stimulus: Carrier API becomes slow or unavailable during checkout shipping rate selection

Environment: 30% of checkout attempts hit degraded carrier

Response: Circuit breaker opens after 3 failures. Fallback cached rates are served. Customer sees estimated rates. Checkout is not blocked.

Measure: Checkout completion rate ≥ 97% during carrier degradation. Circuit breaker opens within 3 failures. Fallback rate served within 100ms.

QAS-4 — Traceability: Order State Visibility Across Channels

Source: Customer support agent or customer self-service

Stimulus: Customer asks for current order status after WMS dispatch event published

Environment: Normal operation, order fulfilled via OpenBoxes WMS

Response: Commerce Core order detail shows updated fulfillment status within acceptable lag of WMS event publication.

Measure: Order status reflects WMS ShipmentDispatched event within 10 seconds of event publication. Status queryable via Commerce Core API with full event trail.

QAS-5 — Recoverability: Stock Reconciliation After WMS Reconnection

Source: OpenBoxes WMS

Stimulus: WMS was offline for 15 minutes during which warehouse picks were processed locally. WMS reconnects and publishes buffered events.

Environment: Post-degradation recovery, message bus reconnect

Response: Inventory Context processes buffered WMS events in order. Commerce Core stock view reconciles. No manual intervention required.

Measure: Full reconciliation completes within 2 minutes of WMS reconnection. No duplicate stock deductions. Idempotency keys prevent double-processing.

5. Chosen Framework: ADD (Attribute-Driven Design)

5.1 Framework Selection

We selected ADD (Attribute-Driven Design) as the primary design framework for this assignment.

5.2 Why ADD Fits Scenario C

ADD is explicitly driven by quality attribute requirements. The architect starts with a prioritized set of quality attribute scenarios and uses them to select architectural drivers, patterns, and tactics at each design iteration. This maps directly to Scenario C for several reasons:

- Scenario C is defined by quality attribute tensions — availability vs. consistency, resilience vs. simplicity, traceability vs. performance — rather than by functional features alone. ADD is designed precisely for this kind of driver.
- ADD's iterative decomposition allows us to keep what works in nopCommerce (the monolith handles order management well) and selectively extract only what the quality attributes demand (Inventory Context for consistency; async bus for availability).
- ADD explicitly forces the team to document why each structural decision was made. This creates natural traceability from QAS to ADR to implementation — exactly what the assignment evaluation requires.
- Compared to TOGAF/ADM, ADD is lighter and more implementation-oriented, which suits the scope and timeline of this assignment. Compared to ACDM, ADD places quality attributes first rather than use-case coverage, which is the right priority for a resilience-focused scenario.

5.3 How We Applied ADD

ADD Step	Application to VerdeMart
Step 1 — Review inputs	Identified business drivers (omnichannel unification), constraints (no full rewrite, no shared DB across extracted boundaries), and concerns (stock races, ERP coupling).
Step 2 — Establish iteration goal	Iteration 1: Address availability (QAS-1, QAS-3). Iteration 2: Address consistency (QAS-2). Iteration 3: Address traceability and recoverability (QAS-4, QAS-5).
Step 3 — Choose element to decompose	Started with the Commerce Core monolith. Identified the Inventory module as the first extraction candidate.
Step 4 — Choose design concepts	Outbox pattern for ERP/WMS publication. Reservation API + optimistic locking for stock consistency. Circuit breaker for carrier resilience. Event-carried state transfer for cross-channel visibility.

Step 5 — Instantiate and allocate	Allocated patterns to concrete components: RabbitMQ bus, Inventory microservice, WireMock carrier surrogate, nopCommerce outbox table.
Step 6 — Sketch views and record decisions	Produced context map, component interaction diagrams, and ADRs.

6. Target Architecture

6.1 Overview

The target architecture keeps nopCommerce as the Commerce Core (modular monolith) and adds a small number of independently deployable services around it, connected by a message bus. The principle is: extract only what the quality attributes demand, and keep the monolith responsible for what it does well.

6.2 Main Components

Component	Role and Rationale
Commerce Core (nopCommerce)	Handles the full order lifecycle, cart, pricing, catalog, and customer-facing web UI. Extended with an outbox table and an event publisher that writes to RabbitMQ after successful database commits.
Inventory Service (extracted)	Standalone ASP.NET Core microservice. Single source of truth for cross-channel stock reservation. Exposes a reservation API (optimistic locking + idempotency keys). Subscribes to WMS fulfillment events.
RabbitMQ Message Bus	Shared async infrastructure. Topics: order.confirmed, shipment.dispatched, stock.adjusted, tracking.updated. Not a shared database — only message routing.
ERPNext (Back-Office)	Receives order summaries asynchronously via RabbitMQ consumer. Does not participate in the synchronous checkout path. Failure is isolated.
OpenBoxes (WMS)	Receives pick assignments via RabbitMQ. Publishes ShipmentDispatched and StockAdjusted events. Represents the warehouse execution context.
OSPOS / POS Surrogate	In-store sales channel. Calls Inventory Service reservation API. Publishes sale events to RabbitMQ.
WireMock (Carrier Surrogate)	Simulates shipping carrier APIs with configurable latency, errors, and rate responses. Allows realistic circuit-breaker testing.
Keycloak (Identity)	Shared identity provider for staff-facing interfaces (admin, WMS, POS). nopCommerce customer auth remains internal for scope reasons.

6.3 Data Ownership

Data Domain	Owner / Storage

Order state	nopCommerce SQL database (authoritative)
Stock levels & reservations	Inventory Service database (own SQL instance — no shared DB)
Financial records	ERPNext database
Warehouse execution state	OpenBoxes database
Message routing	RabbitMQ (infrastructure, not data store)

6.4 Synchronous and Asynchronous Interactions

Synchronous (in the checkout critical path):

- Browser → Commerce Core: HTTP/S, standard checkout flow
- Commerce Core → Inventory Service: Stock reservation API call (REST, optimistic lock, idempotency key)
- Commerce Core → WireMock Carrier: Shipping rate retrieval, protected by circuit breaker with cached fallback

Asynchronous (decoupled, failure-isolated):

- Commerce Core → RabbitMQ: OrderConfirmed event published via outbox pattern after DB commit
- RabbitMQ → ERPNext consumer: Financial record creation, retried on failure, dead-letter queue on exhaustion
- RabbitMQ → OpenBoxes consumer: Fulfillment assignment, processed by WMS pick workflow
- OpenBoxes → RabbitMQ → Inventory Service: StockAdjusted events after warehouse picks
- OpenBoxes → RabbitMQ → Commerce Core: ShipmentDispatched, TrackingUpdated events for order status display

6.5 Cross-Cutting Concerns

Concern	Decision
Idempotency	All message consumers use idempotency keys. Stock reservation API accepts idempotency tokens. Prevents duplicate processing on retry.
Outbox Pattern	Commerce Core writes events to an outbox table in the same DB transaction as the order. A background publisher reads and forwards to RabbitMQ.
Circuit Breaker	Polly circuit breaker on carrier API adapter. Opens after 3 consecutive failures. Serves cached rates in open state. Half-open probing after 30s.

Dead-Letter Handling	Failed ERP and WMS consumers route to DLQ after 3 retries. DLQ is monitored. Manual requeue tooling is in scope.
Structured Logging	Correlation IDs propagated across all services for end-to-end trace reconstruction. OpenSearch used for log aggregation.
Health Checks	ASP.NET Core health endpoints on Commerce Core and Inventory Service. RabbitMQ liveness monitored. Exposed to Docker Compose health-check configuration.

7. Architectural Decision Records

The following ADRs document the major decisions made in this checkpoint. Each includes the context, decision, rationale, and rejected alternatives.

ADR-001: Extract Inventory Management as an Independent Service

Status	Accepted
Date	May 2026
Drivers	QAS-2 (cross-channel stock consistency), P1 (inventory race condition)

Context

nopCommerce manages stock in a single SQL table shared implicitly across all channels. With web and POS channels both reducing stock without a reservation mechanism, overselling is a likely failure mode. The consistency requirement cannot be met by adding application-level locking inside the monolith without degrading checkout throughput.

Decision

Extract stock reservation and availability into a standalone Inventory Service with its own database. The service exposes a reservation API using optimistic concurrency control and idempotency keys. Commerce Core calls this API synchronously during checkout. No other service reads or writes the inventory database directly.

Rationale

Extracting this boundary allows the consistency model (optimistic locking, reservation lifecycle) to evolve independently of the Commerce Core. It also allows the service to be scaled independently if inventory lookup becomes a throughput bottleneck. The no-shared-database constraint is honoured.

Rejected Alternatives

- Application-level locking in nopCommerce: Pessimistic locks in the monolith DB would solve the race condition but create contention and reduce checkout throughput. Rejected.
- Eventual consistency for stock: Accept occasional oversell and compensate. Rejected because oversell creates real operational cost (refunds, customer trust) and the scenario explicitly requires cross-channel consistency.
- POS and Web sharing a queue: Serialise all stock writes through a queue. Adds latency to every checkout. Rejected for checkout UX reasons.

ADR-002: Use Outbox Pattern for Reliable Event Publication

Status	Accepted
Date	May 2026

Drivers	QAS-1 (ERP availability), QAS-5 (recoverability), P2 (synchronous ERP coupling)

Context

Commerce Core must notify ERPNext and OpenBoxes when an order is confirmed. Calling these systems synchronously at checkout creates hard dependencies — ERP downtime blocks checkout. Publishing directly to RabbitMQ after the DB commit risks losing events if the publish fails or the process crashes between the commit and the publish.

Decision

Implement the Outbox Pattern inside nopCommerce. After a successful order commit, an outbox record is written to the same SQL database transaction. A background relay process reads the outbox and publishes to RabbitMQ, marking records as published. This guarantees at-least-once delivery without coupling checkout success to RabbitMQ or ERP availability.

Rationale

The outbox is the minimum reliable mechanism to decouple checkout from ERP/WMS without introducing distributed transaction complexity. It fits naturally inside the nopCommerce monolith without requiring a full message broker SDK in the checkout path.

Rejected Alternatives

- Synchronous ERP call at checkout: Direct HTTP call to ERPNext from the order service. Rejected — ERP downtime fails checkout (QAS-1 violation).
- Dual-write to DB and RabbitMQ: Write to DB and publish to bus in sequence, no outbox. Rejected — if the process crashes between the two writes, events are silently lost.
- Saga orchestrator: Full distributed saga for order confirmation. Rejected as over-engineering for the current scope; outbox with idempotent consumers is sufficient.

ADR-003: Carrier Integration Protected by Circuit Breaker with Cached Fallback

Status	Accepted
Date	May 2026
Drivers	QAS-3 (carrier API degradation), P4 (carrier API fragility)

Context

nopCommerce shipping plugins call carrier APIs synchronously during checkout to retrieve rates. If the carrier API is slow or unavailable, the checkout page hangs or fails. This is a real operational risk during high-traffic periods or carrier maintenance windows.

Decision

Wrap carrier API calls in a Polly circuit breaker. Cache the last known rates per origin/destination/weight band with a TTL of 15 minutes. When the circuit is open, serve cached rates with a visual indicator to the customer. The circuit transitions to half-open after 30 seconds and probes with a single request before closing.

Rationale

The circuit breaker prevents cascading failures where a slow carrier API ties up all checkout threads. Cached fallback rates are a business decision: slightly stale rates are preferable to a failed checkout. The WireMock surrogate allows us to validate this behaviour under controlled degradation conditions.

Rejected Alternatives

- Timeout only (no circuit breaker): A hard timeout stops hangs but does not prevent repeated slow calls. Rejected — each slow call still consumes a thread for the timeout duration.
- Async rate retrieval (pre-checkout): Fetch rates in the background and show them before checkout. Rejected for this iteration — adds complexity to cart UX without proportional benefit at current scale.
- Remove carrier rates from checkout entirely: Show flat-rate estimates. Rejected — business requirement is to show carrier-accurate rates where possible.

ADR-004: Keep Commerce Core as Modular Monolith

Status	Accepted
Date	May 2026
Drivers	Scope constraint, risk management, assignment spirit

Context

There is a risk of over-decomposing nopCommerce into microservices, motivated by architectural fashion rather than quality attribute necessity. The assignment explicitly warns against this and asks for selective evolution with a defensible rationale.

Decision

Keep order management, cart, catalog, pricing, and customer-facing UI inside the nopCommerce monolith. Only extract what explicit quality attribute scenarios demand: Inventory Service (QAS-2 consistency) is the only extracted service in scope for this checkpoint. Further extraction is deferred to future iterations and must be justified by new QAS evidence.

Rationale

The monolith boundary preserves developer productivity, transactional consistency within Commerce Core, and operational simplicity. Distributed systems introduce network failure modes, deployment complexity, and data ownership challenges that are only worth accepting if quality attribute gains are clear and measurable.

Rejected Alternatives

- Full microservices decomposition: Separate services for catalog, orders, pricing, customers. Rejected — no QAS justifies this decomposition at current scale. Adds distributed transaction complexity with no demonstrated benefit.
- Extract Order Service: Separate the order lifecycle from the nopCommerce monolith. Rejected — order management is well-bounded inside nopCommerce and does not create cross-channel consistency problems that cannot be solved with the outbox pattern.

ADR-005: RabbitMQ as the Integration Message Bus

Status	Accepted
Date	May 2026
Drivers	QAS-1, QAS-4, QAS-5 — async coordination, fan-out, retry

Context

The architecture requires reliable async event delivery between Commerce Core, ERPNext, OpenBoxes, and the Inventory Service. A message bus is needed to decouple these systems and support retry, dead-lettering, and fan-out.

Decision

Use RabbitMQ with topic exchanges and durable queues. Each consumer has its own queue bound to relevant topics. Failed messages are dead-lettered after 3 retries with exponential backoff. RabbitMQ management UI and OpenSearch log aggregation provide operational visibility.

Rationale

RabbitMQ is operationally simple, has excellent .NET client support (MassTransit or raw AMQP), and provides the fan-out and dead-letter semantics required by the scenario. It is sufficient for the current event volume and team familiarity is high.

Rejected Alternatives

- Apache Kafka: Better for event sourcing and high-throughput log retention. Rejected for this scenario — operational complexity of Kafka is not justified by the event volume or replay requirements of VerdeMart at this stage.
- Direct HTTP callbacks: Services call each other's webhooks. Rejected — creates tight coupling, no retry guarantee, and cascading failures on receiver downtime.

8. Risk and Validation Plan

8.1 Main Architectural Risks

Risk	Severity	Description	Mitigation
R1 — Inventory Service becomes a bottleneck	High	All checkout and POS calls flow through a single reservation service. If it slows or fails, checkout fails.	Independent deployment, health checks, P99 load test before Part 2.
R2 — Outbox relay lag under load	Medium	High order volume could create outbox backlog, delaying ERP/WMS notification.	Load test outbox relay throughput. Define acceptable lag SLA.
R3 — RabbitMQ single point of failure	Medium	If RabbitMQ is unavailable, all async integrations stall (but checkout still works via outbox).	RabbitMQ with durable queues. Outbox acts as buffer. Retry on reconnect.
R4 — WMS event ordering	Low-Medium	Out-of-order WMS events (StockAdjusted before OrderAssigned) could corrupt inventory state.	Sequence numbers on events. Idempotent consumer with deduplication store.
R5 — Circuit breaker misconfiguration	Low	Circuit breaker thresholds set too aggressively could open the circuit on transient errors, degrading UX unnecessarily.	Tune with WireMock fault injection before Part 2 demo.

8.2 Validation Plan

- QAS-1 (ERP availability): Bring down ERPNext container during checkout load test. Verify orders complete and ERP records are created after recovery.
- QAS-2 (stock consistency): Run concurrent reservation test — 50 simultaneous requests for a single-unit item. Assert exactly 1 success.
- QAS-3 (carrier circuit breaker): Configure WireMock with 5s delay and 503 responses. Verify circuit opens, fallback rates served, checkout completes.
- QAS-4 (order visibility): Place order, trigger WMS ShipmentDispatched event, query Commerce Core order status. Assert update within 10s.

- QAS-5 (WMS reconnect): Pause OpenBoxes container for 2 minutes (simulate offline). Process picks locally. Reconnect. Verify buffered events processed and stock reconciled.

9. Evolution Roadmap

9.1 Phased Evolution

Phase	Content
Phase 0 — Baseline (now)	nopCommerce as-is. Single-channel web store. Synchronous ERP. No event bus. No cross-channel stock.
Phase 1 — Foundation (Part 1 → Part 2)	Add RabbitMQ. Implement outbox in nopCommerce. Deploy Inventory Service. Connect WireMock carrier surrogate. ERP receives async events.
Phase 2 — Cross-Channel (Part 2 demo)	Connect OpenBoxes WMS (fulfillment events). Connect OSPOS POS channel (reservation API). Implement circuit breaker on carrier. Add OpenSearch log aggregation.
Phase 3 — Operational Hardening (post-assignment)	Dead-letter monitoring UI. Outbox relay metrics dashboard. Keycloak identity federation for staff. Automated reconciliation job for stock drift.

9.2 Coexistence During Transition

During Phase 1, the nopCommerce monolith continues to serve web customers without interruption. The Inventory Service is introduced as an additive capability — Commerce Core calls it for new reservations but falls back gracefully if it is unreachable (with a temporary in-monolith stock check as safety net). The outbox is introduced as a new database table with zero change to the existing order processing logic other than the additional write.

9.3 What Must Coexist

- nopCommerce monolith stock table and Inventory Service: During Phase 1, both exist. The inventory service is the authoritative reservation source; the monolith table is kept in sync via events. By Part 2, the monolith's direct stock deduction is removed.
- Synchronous ERP call (legacy plugin) and outbox-based async ERP notification: The legacy synchronous ERP plugin is disabled in Phase 1. Replaced by outbox consumer. No parallel writes.

10. Feasibility Spike

10.1 Spike Goal

The riskiest technical assumption in the architecture is that the Inventory Service can handle concurrent stock reservation requests with correct serialization under realistic checkout load, without becoming a latency bottleneck at the checkout critical path.

10.2 Spike Design

We will implement a minimal Inventory Service with the following:

- ASP.NET Core 8 API with a single endpoint: POST /reservations
- PostgreSQL database with a stock table and a reservations table
- Optimistic concurrency control using a row-version column
- Idempotency key accepted as a header; duplicate requests return the existing reservation result
- Docker Compose deployment alongside a stripped-down nopCommerce instance

10.3 Spike Validation

We will run the following tests against the spike to validate the architectural assumption:

- Correctness test: 50 concurrent POST /reservations for a single SKU with qty=1. Assert exactly 1 success response (HTTP 200) and 49 conflict responses (HTTP 409).
- Latency test: 200 sequential reservation requests. Assert P99 latency ≤ 200ms (QAS-2 measure).
- Idempotency test: Same idempotency key sent 5 times. Assert all 5 return same result with no duplicate reservations.

10.4 Expected Outcome

The spike should confirm that a simple optimistic-locking approach in a well-indexed PostgreSQL table is sufficient for the expected concurrency levels. If the spike reveals unacceptable latency, we will evaluate Redis-based reservation counters as an alternative — which is the main architectural risk the spike is designed to surface before committing to the full design.

10.5 Spike Deliverable

The spike will be committed to the group repository under /spike/inventory-service with a README documenting the test results, observed latencies, and the conclusion. This evidence will be included in the Part 2 evidence pack.